

For this same title the slug Django will produce will be:

- `'this-is-an-article-title-somethings-happened-part-2'`

As you can see this is a lot cleaner, and easier to understand. When this article is published it will appear as:

<http://some.website/articles/this-is-an-article-title-somethings-happened-part-2>

There are many ways we can match this pattern. An obvious one from what we have learned so far would be to match for lower case characters, numbers and the hyphen '-'. This would be represented as `[a-z0-9-]`, there is also a shorter form `[\w-]`. There are many such short forms in RegEx, '`\w`' stands for all lower and upper case characters, all numbers and the underscore, or `[a-zA-Z0-9_]`.

To match an article slug we would now use the following pattern: `^articles/[\w-]+/$`

Extracting Parameters from URLs

Well, all this is great; it allows us to match a range of URLs, but simply matching isn't enough in the end, we also want to be able to change the content of the page based on the matched bits of the URL.

So if we match a URL like `'archive/2014/04/'` we want the view to know that we matched 2014, and 04 so it can filter the articles for that month of that year and display them.

This is in fact very very simple, as simple as simply placing brackets '()' around the bits of the URL that we want the view function to receive! So the pattern for capturing the year, month and day in the URL becomes:

- `^archive/([0-9]{4})/([0-9]{2})/([0-9]{2})/$`

This will capture the year, month and day separately from this URL and pass those along to the view function where these parameters can be used to fetch the appropriate articles. The old `'test_view'` view function we wrote earlier won't work for this, since the view function needs to be capable of receiving these parameters as well. It's as simple as adding additional parameters to the function declaration, one for each value captured. We'll look into this in more detail in the next chapter.

Regular Expressions also allow you to give a name for the data you are capturing; for instance the pattern for capturing the year could be written as:

- `^archive/(?P<year>[0-9]{4})/$`

You'll notice that we have added `'?P<year>'` before the pattern inside the capturing bracket. This explicitly names this capture as the year. There isn't a huge advantage in doing this in most cases other than